DAY 3

ACTIVITY 1

Waterfall model compare with agile and create a simple workflow for an application

1. Waterfall Model

A traditional, linear software development approach.

Characteristics

- Sequential phases: Requirements → Design → Development → Testing → Deployment → Maintenance
- Each phase must be completed before the next begins
- Changes are difficult once development starts
- Best for projects with stable, well-defined requirements

Advantages

- Easy to plan and manage
- Clear documentation
- Works well for predictable projects (e.g., government, manufacturing)

Limitations

- Not flexible for changes
- Testing happens late
- Risk of delivering something different from customer expectations

2. Agile Model

An iterative, flexible approach focused on customer collaboration and frequent delivery.

Characteristics

- Work delivered in small iterations (Sprints)
- Continuous feedback from users
- Changes can be incorporated anytime
- Teams are cross-functional

Advantages

Advantages

- Faster delivery
- High customer satisfaction
- Better risk management
- Adapts to evolving requirements

Limitations

- Needs active customer involvement
- Less predictability for long-term planning
- Requires experienced teams

Simple Workflow for an Application (Agile-style)

Let's assume we are building a simple Task Management Application.

Simple Agile Workflow

1. Requirement Gathering

- Product Owner collects requirements
- Create a Product Backlog
  - Example stories:
    - "As a user, I want to create tasks"
    - "As a user, I want to mark tasks as completed"

2. Sprint Planning

- Team selects backlog items for the 2-week sprint
- Break into tasks
- Define acceptance criteria

3. Design (Quick Wireframes)

- UI mockups for task list, create task form
- Architecture discussion (frontend + backend + database)

4. Development (Iteration)

- Developers build features in small increments
- Example sprint tasks:
  - Create Task API
  - UI form for adding tasks

- Database table creation

- Database table creation

5. Testing

- Tester verifies each story
- Regression testing
- Fix issues

6. Sprint Review

- Demonstrate working features to stakeholders
- Gather feedback

7. Sprint Retrospective

- Team reflects on what went well
- Identify areas for improvement
- Plan changes for next sprint

8. Deployment

- Deploy MVP
- Continue with next sprints for new features

ACTIVITY 2

Agile: what is agile manifesto? Different frameworks? Choose and 2 frameworks understand

1. What Is the Agile Manifesto?

The **Agile Manifesto** (created in 2001 by 17 software experts) is a guiding philosophy for modern software development.
It focuses on **flexibility**, **customer collaboration**, and **continuous improvement**.

Agile Manifesto – 4 Core Values

Agile values:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

Meaning: Agile prioritizes people, adaptability, and quick delivery of usable software.

2. What Are the Different Agile Frameworks?

Agile is an **umbrella term**. Under it, many frameworks provide structure on "how" to implement Agile.

Here are the **most popular Agile frameworks**:

A. Scrum

B. Kanban

C. Extreme Programming (XP)

D. Lean

E. SAFe (Scaled Agile Framework)

F. DevOps

G. Crystal

H. DSDM (Dynamic Systems Development Method)

I. Feature-Driven Development (FDD)

3. Choose Any 2 Frameworks – Explained Clearly

Let's choose the most commonly used ones: **Scrum** and **Kanban**.

Framework 1: SCRUM

Scrum is the most widely used Agile framework.
It works in **time-boxed iterations called Sprints** (usually 2–4 weeks).

Key Roles

- **Product Owner** – decides what to build
- **Scrum Master** – removes blockers, supports team
- **Development Team** – builds the product

Key Events (Meetings)

1. **Sprint Planning** – decide what to do in the sprint
2. **Daily Scrum** – 15-minute stand-up
3. **Sprint Review** – demo to stakeholders
4. **Sprint Retrospective** – internal team improvement

5 of 13

Artifacts

- **Product Backlog** – list of all features
- **Sprint Backlog** – selected work for sprint
- **Increment** – completed working product

When Scrum is best

- Projects needing regular updates
- Teams that prefer time-bound cycles
- When priorities change frequently

Framework 2: KANBAN

Kanban is a **visual workflow method** focused on continuous delivery.

Core Concepts

- Visualize work on a **Kanban board**
  - Columns: To Do → In Progress → Testing → Done
- Limit work-in-progress (**WIP limits**)
- Deliver continuously (no fixed sprints)

Principles

- Start with what you do now
- Make workflow visible
- Improve flow
- Review processes regularly

When Kanban is best

- Support teams (IT support, maintenance)
- Continuous, incoming work
- Projects where tasks vary in size and priority

ACTIVITY 3

Tools used in Scrum process/Agile process, any AI tool used in the Agile process and get knowledge about that AI tool

Scrum and Agile teams typically rely on a combination of **project management**, **collaboration**, **documentation**, and **automation tools**

 teams typically rely on a combination of **project management**, **collaboration**, **documentation**, and **automation tools**.
Common categories and tools include:

A. Backlog & Sprint Management Tools

- **Jira** (most widely used)
- **Azure DevOps**
- **ClickUp**
- **Asana**
- **Wrike**

These tools manage backlog items, track sprint progress, and give dashboards for Agile planning. (Examples from top Agile PM tool lists include **Wrike, ClickUp, Asana**, etc.)

B. Collaboration & Communication Tools

- **Microsoft Teams**
- **Slack**
- **Confluence** (documentation)
- **Notio**

C. Agile Ceremony Tools

Used for retrospectives, estimation, workshops:

- **Miro** (for retros, brainstorming)
- **Retrium** (retrospectives)
- **Planning Poker apps**

D. CI/CD & DevOps Tools

- **GitHub / GitLab / Bitbucket**
- **Jenkins**
- **Azure DevOps Pipelines**

These support continuous integration and deployment.

2. AI Tools Used in Agile / Scrum

AI is increasingly used to automate meetings, suggest sprint estimations, detect risks, and improve team flow. Recent sources list several AI tools for Scrum Masters and Agile teams.

Most Recognized AI Tools in Agile Today

1. Atlassian Jira AI (Rovo)

- Automates backlog refinement
- Provides AI-driven insights about sprint health
- Predicts bottlenecks

- Automates backlog refinement
- Provides AI-driven insights about sprint health
- Predicts bottlenecks
- Summarizes tickets and improves backlog grooming

2. Plandek

- Provides performance metrics (DORA, flow)
- Helps optimize velocity and predict delivery

3. Otter.ai / Fireflies.ai

- Automates standup + meeting transcription
- Creates summaries and action items
- Flags blockers

4. Miro AI

- Brainstorming, refinement, prototyping
- Generates retrospective summaries

5. Stepsize AI

- Generates sprint reports
- Tracks technical debt

6. Sally AI

- Turns meetings into structured outputs
- Automates synthesis of decisions, blockers, and risks

7. ClickUp AI, Asana AI, monday.com AI

- Automated task generation
- Predictive timelines
- Workload balancing and risk detection

Deep Dive: Understanding Two AI Tools in Agile

Below are **two AI tools explained simply**, as you can mention in interviews.

What it does

- Adds smart AI features inside Jira
- Summarizes issues

- Helps refine backlog items
- Predicts sprint risks using past data
- Gives insights to reduce grooming friction [sally.io]

Why it is useful

- Saves time during refinement
- Reduces manual writing of ticket descriptions
- Highlights dependencies and risks early
- Improves planning accuracy

Where it is used

- Sprint planning
- Daily standup insights
- Backlog grooming
- Reporting to stakeholders

This is one of the most impactful AI tools as most teams already use Jira.

AI Tool 2: Plandek

What it does

- Analyzes team performance metrics
- Tracks flow efficiency, cycle time, DORA metrics
- Provides predictive indicators for delivery risk [scrum.org]

Why it is useful

- Helps Scrum Masters measure **true team performance**
- Identifies bottlenecks (blocked tasks, slow reviews, etc.)
- Supports continuous improvement with data
- Improves sprint retrospectives with real evidence

Where it is used

- Sprint retros
- Engineering productivity analysis
- Delivery forecasting
- Coaching the team for improvement

ACTIVITY 4

WHITE BOX/ GREY BOX/ BLACK BOX TESTING

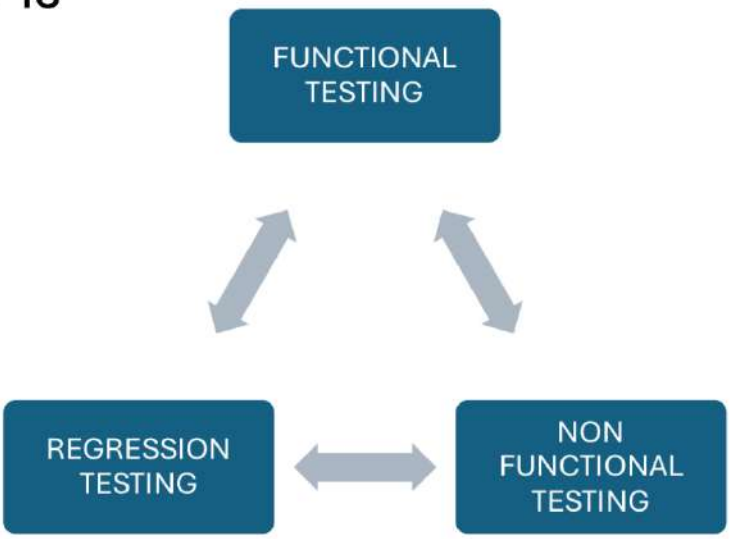
WHITE BOX TESTING

White box testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application.

- The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- White box testing is also known as Structural Testing or Code-based Testing, and it is used to test the software's internal logic, flow, and structure.
- The tester creates test cases to examine the code paths and logic flows to ensure they meet the specified requirements.
- Tools commonly used for whitebox testing include
- **JUnit**: A widely used testing framework for Java applications.
- **NUnit**: A testing framework for .NET applications.
- **TestNG**: A testing framework inspired by JUnit, designed for test configuration and parallel execution.
- **Cucumber**: A tool for behaviour-driven development (BDD) that allows writing tests in plain language.
- **SonarQube**: A tool for continuous inspection of code quality and security vulnerabilities.

BLACK BOX TESTING

Black-box testing is a Type of Software Testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.



Functional Testing:
Functional testing is a type of black box testing where the system is tested based on functional requirements and specifications. In this testing, the tester checks whether each feature of the application works as expected according to the requirement document. The focus is on inputs and outputs without knowing the internal code. It ensures that the system performs the correct functions, such as login, registration, search, and other user operations.

Regression Testing:
Regression testing is performed to make sure that recent changes, bug fixes, or updates in the software have not affected the existing functionality of the application. After modifying the code, testers re-test the already working features to confirm that they still work properly. The main goal of regression testing is to maintain system stability after updates or enhancements.

Non-Functional Testing (NFT):
Non-functional testing focuses on testing the non-functional aspects of the system such as performance, speed, security, usability, reliability, and scalability. Instead of checking what the system does, it checks how well the system performs. For example, it verifies how fast the application loads, how it behaves under heavy traffic, and whether user data is secure

Tools Used :

- **Selenium**: An open-source tool for automating web applications for testing purposes.
- **QTP/UFT (Quick Test Professional/Unified Functional Testing)**: A commercial tool for functional and regression testing.
- **LoadRunner**: A performance testing tool that simulates virtual users to test application performance.
- **Postman**: A tool for testing APIs by sending requests and validating responses.

- **TestComplete**: An automated UI testing tool for desktop, web, and mobile applications.

- **TestComplete**: An automated UI testing tool for desktop, web, and mobile applications.

GREY BOX TESTING

Gray Box Testing is a software testing technique that combines elements of the Black Box Testing technique and the White Box Testing technique. Gray box testing is also known as translucent testing or API testing.

Objectives of Gray Box Testing:

1. To provide the combined advantages of both black box testing and white box testing.
2. To combine the input of developers as well as testers.
3. To improve overall product quality.
4. To reduce the overhead of the long process of functional and non-functional testing.
5. To provide enough free time to developers to fix defects.
6. To test from the user's point of view rather than a designer's point of view.

Tools used :

- **Burp Suite**: A security testing tool for web applications that allows for both black box and grey box testing.
- **OWASP ZAP (Zed Attack Proxy)**: An open-source tool for finding vulnerabilities in web applications.
- **JMeter**: A performance testing tool that can be used for both functional and load testing, with some knowledge of the internal workings of the application.
- **SoapUI**: A tool for testing web services and APIs, allowing for both functional and security testing.

ACTIVITY 5

EQUIVALENCE PARTITIONING (EP)

Input Condition	Valid Partition	Invalid Partition
Card validity	Valid card	Expired / Blocked card
PIN	Correct	Incorrect
Withdrawal amount	Between minimum & maximum	Less than minimum / More than maximum / Not multiple of 100
Account balance	Sufficient	Insufficient

TC No	Description	Input	Expected Result
EP01	Valid card, valid PIN, valid amount	Card OK, PIN Correct, Amount = ₹500	Transaction successful
EP02	Invalid/expired card	Card Expired	Card rejected
EP03	Valid card, incorrect PIN	PIN Wrong	PIN retry message
EP04	Valid card, amount < minimum	Amount = ₹50	Error: Minimum withdrawal is ₹100
EP05	Valid card, insufficient balance	Amount = ₹10,000, Balance = ₹2,000	Error: Insufficient balance

Boundary Value Analysis (BVA)

TC No	Boundary	Input Amount	Expected Result
BVA01	Minimum Boundary	₹100	Success
BVA02	Below Minimum	₹99	Error: Minimum is ₹100
BVA03	Just Above Minimum	₹101	Error: Amount must be a multiple of 100
BVA04	Maximum Limit	₹10,000	Success
BVA05	Above Maximum	₹10,001	Error: Exceeds maximum limit

Decision Table Testing

TC No	Card	PIN	Balance	ATM Cash	Output
DT01	Valid	Correct	Sufficient	Yes	Cash Dispensed
DT02	Valid	Incorrect	–	Yes	PIN Error
DT03	Invalid	–	–	Yes	Card Rejected
DT04	Valid	Correct	Insufficient	Yes	Low Balance Error
DT05	Valid	Incorrect (3 times)	–	Yes	Card Blocked